Reducibility 06

Thumrongsak Kosiyatrakul
tkosiyat@cs.pitt.edu

Reductions

- Recall the concept of reducibility:
  - A problem A is reducible to a problem B if the solution to the problem B can be used to solve the problem A
- To show that a language A is undecidable, we show that A_{TM} (or other undecidable language) is reducible to A
- A_{TM} is reducible to $HALT_{\mathrm{TM}}$
  - We want to know whether $\langle M, w \rangle \in A_{\mathrm{TM}}$
  - Turn $\langle M, w \rangle$ to $\langle M', w' \rangle$ (in our case $w' = \varepsilon$)
  - Check whether $\langle M', w' \rangle \in HALT_{\mathrm{TM}}$
  - $\langle M, w \rangle \in A_{\mathrm{TM}} \leftrightarrow \langle M', w' \rangle \in HALT_{\mathrm{TM}}$
- A_{TM} is reducible to E_{TM}
  - We want to know whether $\langle M, w \rangle \in A_{\mathrm{TM}}$
  - Turn $\langle M, w \rangle$ to $\langle M' \rangle$
  - Check whether $\langle M' \rangle \in E_{\mathrm{TM}}$
  - $\langle M, w \rangle \in A_{\mathrm{TM}} \leftrightarrow \langle M' \rangle \notin E_{\mathrm{TM}}$

Reductions

- A_{TM} is reducible to $REGULAR_{\mathrm{TM}}$
  - We want to know whether $\langle M, w \rangle \in A_{\mathrm{TM}}$
  - Turn $\langle M, w \rangle$ to $\langle M' \rangle$
  - Check whether $\langle M' \rangle \in REGULAR_{\mathrm{TM}}$
    - $\langle M, w \rangle \in A_{\mathrm{TM}} \leftrightarrow \langle M' \rangle \in REGULAR_{\mathrm{TM}}$
    - $\langle M, w \rangle \in A_{\mathrm{TM}} \leftrightarrow \langle M' \rangle \notin REGULAR_{\mathrm{TM}}$
- A_{TM} is reducible to EQ_{TM}
  - We want to know whether $\langle M, w \rangle \in A_{\mathrm{TM}}$
  - Turn $\langle M, w \rangle$ to $\langle M_1, M_2 \rangle$
  - Check whether $\langle M_1, M_2 \rangle \in EQ_{\mathrm{TM}}$
  - $\langle M, w \rangle \in A_{\mathrm{TM}} \leftrightarrow \langle M_1, M_2 \rangle \in EQ_{\mathrm{TM}}$
- E_{TM} is reducible to EQ_{TM}
  - We want to know whether $\langle M \rangle \in E_{\mathrm{TM}}$
  - Turn $\langle M \rangle$ to $\langle M, M_{\mathrm{E}} \rangle$
  - Check whether $\langle M, M_{\mathrm{E}} \rangle \in EQ_{\mathrm{TM}}$
  - $\langle M \rangle \in E_{\mathrm{TM}} \leftrightarrow \langle M, M_{\mathrm{E}} \rangle \in EQ_{\mathrm{TM}}$

Reductions via Computation Histories

- So for we construct TMs to show that A_{TM} is reducible to certain languages
- Another method is **Reduction via Computation Histories**
- Generally use when the problem to be shown undecidable involves testing for the **existence of something**
  - Given a polynomial p, does it has an **integral root**?
  - Given a set of dominoes, does it has a match?

Definitions

- Recall definitions about configuration:
  - Suppose the start state of TM M is q_0, the **start configuration** of M on input w is q_0w
  - An **accepting configuration** is a configuration $uq_{accept}v$ for strings u and v
  - A **rejecting configuration** is a configuration $uq_{reject}v$ for strings u and v
  - A configuration C_i **legally** go to C_j according to the transitions of the TM M

Reductions via Computation Histories

Definition 5.5

Let M be a Turing machine and w an input string. An **accepting computation history** for M on w is a sequence of configurations, $C_1, C_2, \ldots C_l$ where C_1 is the start configuration of M on w, C_l is an accepting configuration of M, and each C_i legally follows from C_{i-1} according to the rules of M. A **rejecting computation history** for M on w is defined similarly, except that C_l is a rejecting configuration.

- Note that M must halt on input w. Otherwise, **accepting** or **rejection** computation history does not exists
- For simplicity, we only consider **deterministic** Turing machine
  - If M halts on w, **exactly one** computation history exists

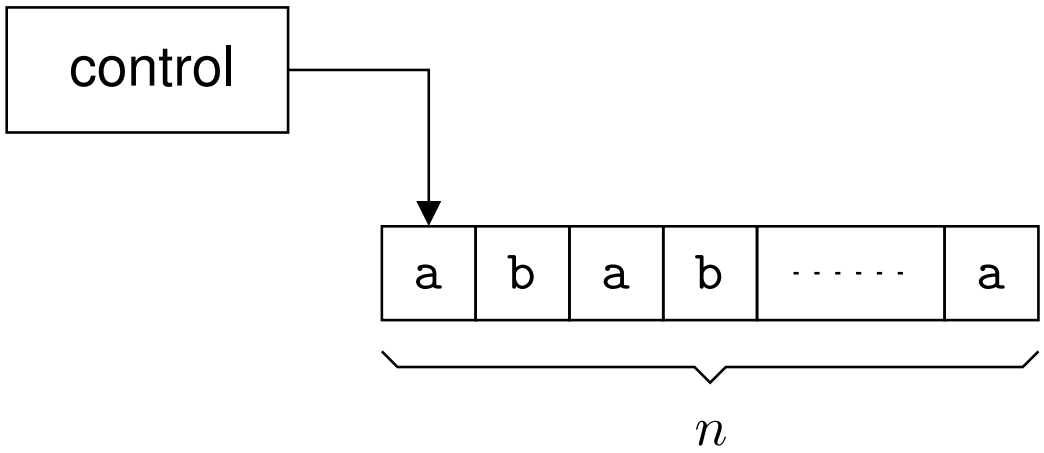
Reductions via Computation Histories

- Again, to show that a language A is undecidable, we show that A_{TM} is reducible to A
- Given a TM M and a string w, there is no algorithm to check whether $\langle M, w\rangle \in A_{\mathrm{TM}}$ because A_{TM} is undecidable
- But we know that
  - If M accepts w, $\langle M, w\rangle \in A_{\mathrm{TM}}$
  - If M does not accept w, $\langle M, w\rangle \notin A_{\mathrm{TM}}$
- Using the concept of computation histories:
  - If the accepting computation history of M on input w exists $\rightsquigarrow$ M accepts w $\rightsquigarrow$ $\langle M, w\rangle \in A_{\mathrm{TM}}$
  - If the accepting computation history of M on input w does not exists $\rightsquigarrow$ M does not accept w $\rightsquigarrow$ $\langle M, w\rangle \notin A_{\mathrm{TM}}$
- Let's try to use reduction via computation histories with a variation of a TM called **Linear Bounded Automaton** (LBA)

Linear Bounded Automaton

Definition 5.6

A **linear bounded automaton** is a restricted type of Turing machine wherein the tape head isn't permitted to move off the portion of the tape containing the input. If the machine tries to move its head off either end of the input, the head stays where it is.



- With an input of length n, the amount of memory available is **linear** in n.

Linear Bounded Automaton

- **Problem**: Given an LBA M and a string w, is there an algorithm to determine whether M accepts w?
- Formally,

$$A_{\text{LBA}} = \{\langle M, w\rangle \mid M \text{ is an LBA that accepts } w\}$$

- Is A_{LBA} decidable?
- This sounds like A_{TM}

$$A_{\text{TM}} = \{\langle M, w\rangle \mid M \text{ is a TM that accepts } w\}$$

- Even though A_{TM} is undecidable, A_{LBA} is actually decidable

Linear Bounded Automaton

- Recall a configuration uqv
  - uv is the content of the tape
- Recall that very TM has an infinite long tape
- Therefore, there are infinite number of distinct strings uv
- But the tape of an LBA is finite
  - The length of its tape on input string w is $|w|$
  - The number of states and $|\Gamma|$ of an LBA are also finite
  - Because of these, the set of all distinct configurations that an LBA can have on an input string w is finite

Linear Bounded Automaton

- Consider an LBA with the following specification:
  - The number of states is q
  - $|\Gamma| = g$ (number of symbols in its tape alphabets)
  - The length of the tape is n
- From the above LBA:
  - For each square on the tape, there can be one of g symbols
    - There are the total of g^n unique tape contents
  - There are n possible positions of the tape head
  - For each tape head position, it can be in one of q states
- Thus, there are exactly qng^n distinct configurations
- Because of this, we can detect when an LBA is entering a loop by checking whether sequence of configurations when running an LBA and a string w contains a duplicate

Linear Bounded Automaton

- To detect an infinite loop in an LBA, consider an LBA M where its series of configuration while processing w is as follows:

$$C_1, C_2, C_3, C_4, C_5, C_6, C_7, C_8, C_9, C_{10}, C_{11}, \ldots$$

- What if the configuration C_3 is the same as C_9?
  - The content of the tape are exactly the same
  - The current state of M are the same
  - The tape head are at the same location
- Thus
  - C_{10} will be the same as C_4
  - C_{11} will be the same as C_5
  - and so on
- However, this method requires us to keep track of sequence of configurations

Linear Bounded Automaton

- Recall that if we run an LBA on input w for 1 step, we already sees two configurations
  - The start configuration
  - The next configuration that legally goes from the start configuration
- Thus, if we run an LBA on input w for k steps, we already see $k + 1$ configurations
- Recall that the total number of distinct configurations is qng^n, we can simply run the LBA for qng^n steps or until it halts
  - If it does not halt after qng^n steps, we already see $qng^n + 1$ configurations
  - But there are only qng^n distinct configurations
  - Thus, at least two of them must be the same (loop indefinitely)

Linear Bounded Automaton

Theorem 5.9

$A_{\text{LBA}} = \{\langle M, w\rangle \mid M \text{ is an LBA that accepts string } w\}$ is decidable.

- Construct a TM L that decides A_{LBA} as follows:

  $L =$ "On input $\langle M, w\rangle$, where M is an LBA and w is a string:

  1. Simulate M on input w for qng^n steps or until it halts.
  2. If M has halted, $accept$ if it has accepted and $reject$ if it has rejected. if it has not halted, $reject$."

E_{LBA} is Undecidable

- Now, we just verify that A_{LBA} is decidable
- But what about the problem of determining whether an LBA M accepts no strings ($L(M) = \emptyset$)
- Formally,

$$E_{\mathrm{LBA}} = \{\langle M \rangle \mid M \text{ is an LBA and } L(M) = \emptyset\}$$

- It turns out that this problem is unsolvable
- In other words, E_{LBA} is undecidable
- Let's try to prove it using the same method that we used to prove that E_{TM} is undecidable to try to prove that E_{LBA} is undecidable

E_{LBA} is Undecidable

Assume that E_{LBA}. Since E_{LBA} is decidable, there exists a TM R that decides E_{LBA}. Construct a TM S (to decide A_{TM}) as follows:

$S =$ "On input $\langle M, w \rangle$ where M is a TM and w is a string:

1. Construct an LBA M' as follows:

   $M' =$ "On input x:

   1. Run M on input w.
   2. If M accepts w, *accept*; otherwise, *reject*."
2. Run R (a decider for E_{LBA}) on input $\langle M' \rangle$.
3. If R accepts $\langle M' \rangle$, *reject*. If R rejects $\langle M' \rangle$, *accept*."

Next, we need to prove that the TM S above, decides A_{TM}...

Did you see a problem in the above prove?

- M' (constructed by S) must be an LBA
- An LBA has a finite long tape
- To run TM M on input w, the LBA M' may need infinite long tape

You cannot construct an LBA that can always run a TM M on an input w

E_{LBA} is Undecidable

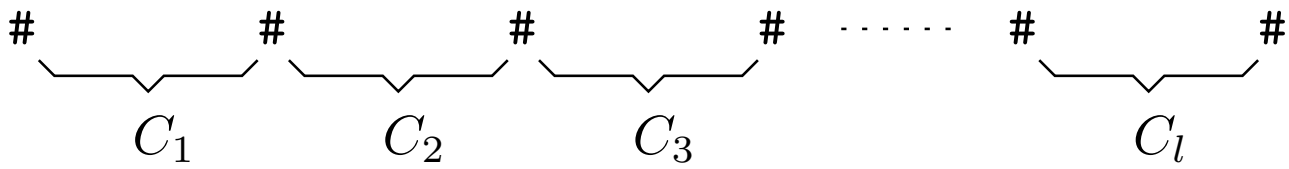
- In this case, we need to show that A_{TM} is reducible to E_{LBA} using the **computation history** method
- This method will also give you the same conclusion:
  - If E_{LBA} is decidable, A_{TM} is decidable
- The idea is to construct an LBA B that accepts exactly one string
- The string will be a string representation of the **accepting computation history** for M on w
- Why this work?
  - If $\langle M, w \rangle \in A_{\text{TM}} \rightsquigarrow M$ accepts $w \rightsquigarrow$ the accepting computation history for M on w exists $\rightsquigarrow$ LBA B accepts exactly one string $\rightsquigarrow L(B) \neq \emptyset \rightsquigarrow \langle B \rangle \notin E_{\text{LBA}}$
  - If $\langle M, w \rangle \notin A_{\text{TM}} \rightsquigarrow M$ does not accept $w \rightsquigarrow$ the accepting computation history for M on w does not exists $\rightsquigarrow$ LBA B accepts no strings $\rightsquigarrow L(B) = \emptyset \rightsquigarrow \langle B \rangle \in E_{\text{LBA}}$

E_{LBA} is Undecidable

- But can we construct an LBA that accepts exactly one string which is the accepting computation history for M on w without requiring an infinite amount of tape?
- Recall that an LBA cannot always runs M on input w because of its finite long tape
- So, can an LBA checks whether a given string is the accepting computation history for M on w without actually run M on input w?
- It turns out, we can

E_{LBA} is Undecidable

- An accepting computation history for M on w is a series of configurations $C_1, C_2, \ldots C_l$ where
  - C_1 is the start configuration of M on input w
  - C_l is an accepting configuration
  - C_i yields C_{i+1} according to M
- We can represents an accepting computation history by a single string with the configurations separated from each other by the # symbol



- For example an accepting computation history of TM M_2 in Example 3.7 on input 0 can be represented as a single string as follows:

$$\#q_1 0 \# \sqcup q_2 \# \sqcup \sqcup q_{\text{accept}} \#$$

E_{LBA} is Undecidable

- With an input string x to the LBA B, the machine B must check the following conditions before accepting x
  1. x is in the form of $\#C_1\#C_2\#\ldots\#C_l\#$ for a number l
  2. C_1 is the start configuration for M on w
  3. C_{i+1} legally follows from C_i
  4. C_l is an accepting configuration for M
- Given the definition of a TM M and a input string w:
  - We know the start state of M
    - Thus, we know the start configuration
  - We know the transition function of M
    - Thus, we can check whether C_{i+1} legally follows from C_i
  - Accepting configuration is just $uq_{accept}v$ for strings u and v
- This show that there is an algorithm to check whether an input string x is an accepting computation history for M on w without actually running M on input w
  - Therefore, the LBA B can be constructed

E_{LBA} is Undecidable

Theorem 5.10

$E_{\mathrm{LBA}} = \{\langle M \rangle \mid M \text{ is an LBA and } L(M) = \emptyset\}$ is undecidable.

- For the sake of contradiction, assume that E_{LBA} is decidable
- There exists a TM R decides E_{LBA}
- Construct a TM S decides A_{TM} as follows:

  $S =$ "On input $\langle M, w \rangle$, where M is a TM and w is a string:

  1. Construct LBA B from M and w as described in the proof idea
  2. Run R on input $\langle B \rangle$
  3. If R rejects $\langle B \rangle$, *accept*; if R accepts $\langle B \rangle$, *reject*."

E_{LBA} is Undecidable

- Sketch of the proofs:
  - If $\langle M, w \rangle \in A_{\mathrm{TM}}$:
    - $\rightsquigarrow$ M accepts w
    - $\rightsquigarrow$ The accepting computation history for M on w exists
    - $\rightsquigarrow$ B accepts exactly one string
    - $\rightsquigarrow$ $L(B) \neq \emptyset$
    - $\rightsquigarrow$ R rejects $\langle B \rangle$
    - $\rightsquigarrow$ S accepts $\langle M, w \rangle$
  - If $\langle M, w \rangle \notin A_{\mathrm{TM}}$:
    - $\rightsquigarrow$ M does not accept w
    - $\rightsquigarrow$ The accepting computation history for M on w does not exists
    - $\rightsquigarrow$ B accepts no strings
    - $\rightsquigarrow$ $L(B) = \emptyset$
    - $\rightsquigarrow$ R accepts $\langle B \rangle$
    - $\rightsquigarrow$ S rejects $\langle M, w \rangle$
- This show that S is a decider for A_{TM}
- In other words, A_{TM} is decidable — contradiction
- Therefore, E_{LBA} is undecidable